

PROJETO 2

Detecção de Objetos em Imagens de Satélite com YOLO

Edição São Paulo — região da PUC-SP (Perdizes e arredores)

Projeto de ponta a ponta — coleta programática, anotação, treino, avaliação e inferência

Disciplina	Aprendizagem de Máquina
Avaliação	P2 — Projeto Final
Modalidade	Grupo de até 5 alunos
Framework	Ultralytics YOLOv8 / YOLOv11 (PyTorch)
Ambiente de treino	Google Colab Free (GPU T4)
Fonte de imagens	ESRI World Imagery (XYZ tiles públicos) + Google Earth Web (complementar)
Recorte geográfico	Perdizes, Higienópolis, Pacaembu, Sumaré, Pinheiros, Itaim e Av. Paulista
Notebooks de apoio	Projeto_P2_Mosaico_Perdizes.ipynb (z=18) e Projeto_P2_Mosaico_Perdizes_HIRES.ipynb (z=19)
Entrega	Dataset anotado + Notebook (.ipynb) + Pesos (.pt) + Relatório + Apresentação

1. Objetivo

O objetivo deste projeto é vivenciar, na prática, o ciclo completo de um sistema de Visão Computacional baseado em deep learning, do dado bruto ao modelo treinado. O grupo será responsável por todas as etapas: definir o objeto-alvo, coletar imagens reais, anotar as caixas delimitadoras, pré-processar o dataset, treinar uma rede de detecção e avaliar os resultados em imagens inéditas.

A mensagem pedagógica central é: cerca de 80% do esforço em um projeto de IA está nos dados, e não na arquitetura. O modelo (YOLOv8/YOLOv11) será praticamente o mesmo para todos os grupos — o que diferencia os resultados é a qualidade do dataset construído.

Nesta edição, o recorte geográfico é a cidade de São Paulo, com foco nos bairros próximos ao campus da PUC-SP em Perdizes. Os grupos usarão como fonte principal de imagens o serviço público de tiles XYZ ESRI World Imagery, que oferece cobertura submétrica de SP (até ~0,27 m/pixel) com licença permissiva para uso educacional (mediante atribuição). Notebooks de apoio prontos para Colab serão disponibilizados pelo professor.

Pipeline do Projeto — Detecção de Objetos em Imagens de Satélite

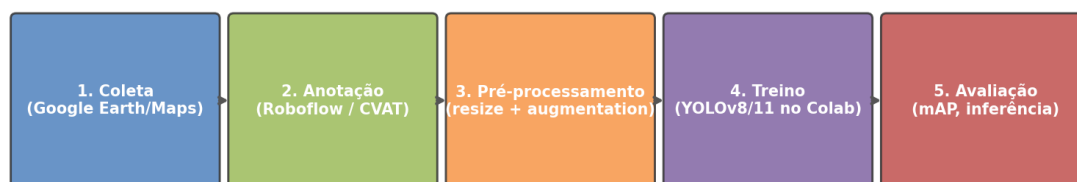


Figura 1 — Visão geral do pipeline do projeto (5 etapas).

2. Dataset — Coleta e Definição do Alvo

Cada grupo deverá construir o próprio dataset a partir de imagens aéreas/ortofotos da cidade de São Paulo. NÃO será usado nenhum conjunto pronto. As imagens serão salvas em .jpg ou .png — não trabalharemos com arquivos .tif/.tiff de sensores nesta etapa.

2.1. Escolha do objeto-alvo (sugestões para São Paulo)

O grupo deve escolher UM único objeto-alvo (uma única classe). Os alvos abaixo foram selecionados por serem abundantes na região da PUC-SP, terem forma geométrica clara em vista superior e oferecerem desafios pedagógicos interessantes:

- **Helipontos em coberturas (RECOMENDADO).** São Paulo é a cidade com a maior frota de helicópteros urbanos do mundo. O **H** branco em círculo é um padrão visual quase único da cidade. Coleta concentrada em **Av. Paulista, Faria Lima, Berrini, Itaim, Vila Olímpia e Brooklin** (todos a ≤ 6 km da PUC-SP).
- Piscinas em coberturas e quintais. Abundantes em **Perdizes, Higienópolis, Pacaembu, Sumaré e Pinheiros**. Caso real: a Receita Federal já usou detecção de piscinas em fotos aéreas para cruzar com declarações de IPTU/IR.
- Quadras de tênis de saibro. Cor laranja característica, formato padronizado. Clubes da zona oeste (Paulistano, Pinheiros, Tietê, Hebraica) e coberturas.
- Campos de futebol society (grama sintética). Verde uniforme que destoa do entorno. Comuns na zona oeste e Marginal Pinheiros.
- Postos de combustível. Cobertura retangular branca sobre as bombas, layout padrão, espalhados por toda a cidade.

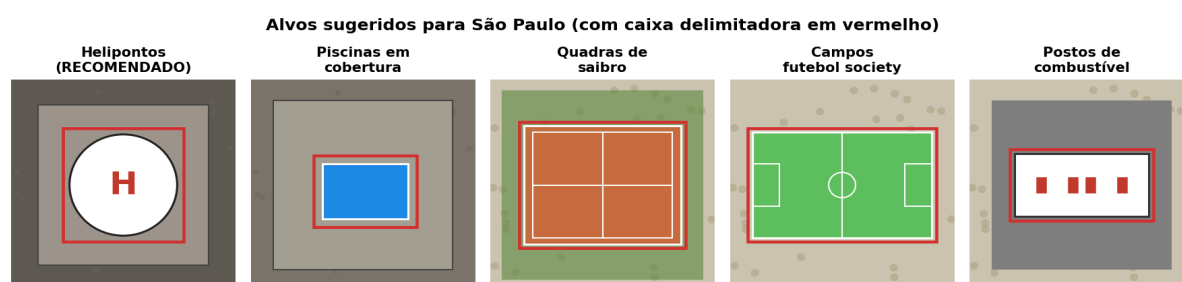


Figura 2 — Alvos sugeridos para a edição São Paulo deste projeto.

Evitar: **objetos ambíguos** (ex.: árvores, casas comuns, carros) — eles geram muitos falsos positivos e desmotivam o grupo. É permitido escolher um alvo **diferente das sugestões acima**, desde que seja aprovado pelo professor antes do início da coleta.

2.2. Coleta programática via ESRI World Imagery (recomendado)

A coleta de imagens será feita pelo serviço público de tiles XYZ ESRI World Imagery (server.arcgisonline.com/ArcGIS/rest/services/World_Imagery/MapServer). **Vantagens decisivas em relação a screenshots manuais: cobertura submétrica para SP (0,55 m/pixel em z=18 e 0,27 m/pixel em z=19, o máximo prático para esta região), licença permissiva para uso educacional mediante atribuição, e — sobretudo — a possibilidade de varrer um bairro inteiro em segundos, gerando um mosaico de tiles padronizados (256×256 px cada).**

Nota sobre o GeoSampa. A Prefeitura de São Paulo disponibiliza ortofotos de altíssima resolução (10 cm/px em áreas urbanas) em geosampa.prefeitura.sp.gov.br, com licença permissiva e sem necessidade de cadastro. O acesso, porém, é por download manual no portal (não há API pública), o que está fora do escopo do projeto base.

Grupos interessados em usar essa fonte como diferencial podem consultar o documento `GeoSampa_passo_a_passo.md` disponibilizado à parte.

Coleta programática — varrendo um bairro via ESRI World Imagery (XYZ tiles)

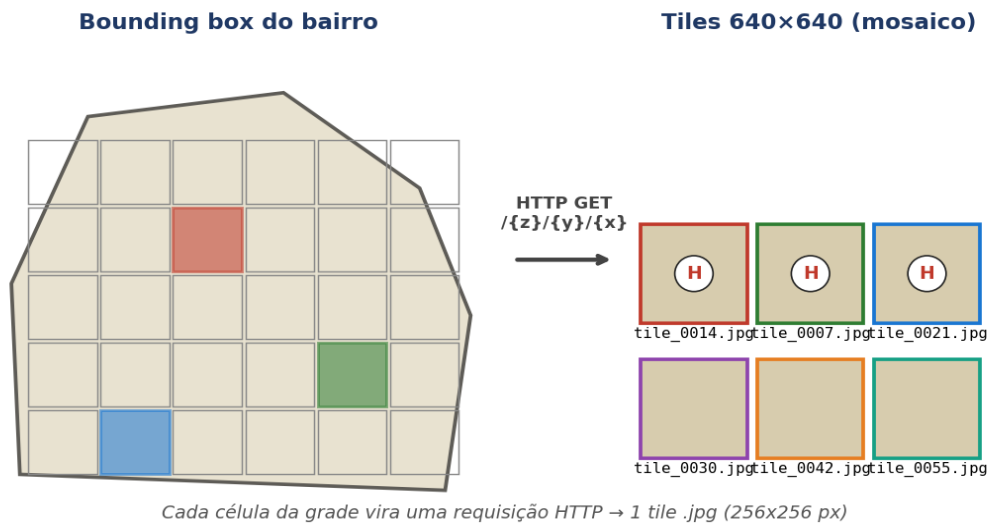


Figura 3 — Coleta programática: o bbox do bairro é convertido em uma grade de tiles XYZ; cada célula vira uma requisição HTTP individual.

Procedimento esperado (apoio: notebooks Projeto_P2_Mosaico_Perdizes.ipynb e Projeto_P2_Mosaico_Perdizes_HIRES.ipynb):

- Escolha o **zoom** conforme o alvo: **z=18** (~0,55 m/px) para piscinas, quadras e campos; **z=19** (~0,27 m/px) para helipontos e alvos menores.
- Defina o **bounding box** do(s) bairro(s) a varrer, no formato (lon_min, lat_min, lon_max, lat_max). Ex.: Perdizes ≈ (-46.685, -23.545, -46.665, -23.530); Itaim ≈ (-46.685, -23.590, -46.665, -23.575); Paulista ≈ (-46.660, -23.572, -46.640, -23.555).
- Rode o script abaixo (ou os notebooks de apoio já com saídas) — ele converte o bbox em tiles XYZ e salva cada um como `tile_z{z}_x{x}_y{y}.jpg`.

```
import math, os, requests
from concurrent.futures import ThreadPoolExecutor

URL = ('https://server.arcgisonline.com/ArcGIS/rest/services/'
      'World_Imagery/MapServer/tile/{z}/{y}/{x}')
HEADERS = {'User-Agent': 'PUC-SP-AM-Aula-Educacional/1.0'}

BBOX = (-46.685, -23.545, -46.665, -23.530) # Perdizes
ZOOM = 18 # use 19 se o alvo for heliponto
OUT = 'mosaico_perdizes'; os.makedirs(OUT, exist_ok=True)

def deg2tile(lat, lon, z):
    n = 2.0 ** z
    x = int((lon + 180.0) / 360.0 * n)
    lat_rad = math.radians(lat)
    y = int((1.0 - math.asinh(math.tan(lat_rad)) / math.pi) / 2.0 * n)
    return x, y

lon0, lat0, lon1, lat1 = BBOX
x_min, y_max = deg2tile(lat0, lon0, ZOOM)
x_max, y_min = deg2tile(lat1, lon1, ZOOM)
```

```
def baixar(args):
    z, x, y = args
    p = f'{OUT}/tile_z{z}_x{x}_y{y}.jpg'
    if os.path.exists(p): return
    r = requests.get(URL.format(z=z, x=x, y=y), headers=HEADERS,
timeout=20)
    if r.status_code == 200 and len(r.content) > 2521:      # filtra
placeholder
        open(p, 'wb').write(r.content)

jobs = [(ZOOM, x, y) for x in range(x_min, x_max+1) for y in range(y_min,
y_max+1)]
with ThreadPoolExecutor(max_workers=4) as ex:
    list(ex.map(baixar, jobs))

print(f'{len(jobs)} tiles em {OUT}/')
```

- **Atribuição obrigatória:** inclua “Source: Esri, Maxar, Earthstar Geographics, and the GIS User Community” em qualquer figura, slide ou relatório que reproduza os tiles ou o mosaico.
- **Curadoria manual obrigatória:** após gerar o mosaico, cada integrante deve revisar os tiles e **descartar os que não têm objetos do alvo** (ex.: tiles cobrindo só telhados sem heliponto, parques, vias). Esta curadoria faz parte da avaliação.
- Cada bbox de bairro deve ser **documentado em uma planilha** (bairro, zoom usado, integrante responsável, nº de tiles gerados, nº de tiles aprovados).

2.3. Coleta manual via Google Earth Web (complementar)

A coleta manual via Google Earth Web fica reservada a alvos pontuais que o aluno identificar navegando pela cidade e que queira incorporar ao dataset. NÃO use para volume — os termos do Google restringem screenshots em massa e o IP é bloqueado após algumas centenas de requisições.

- Manter o mesmo nível de zoom em todas as capturas (escala consistente).
- Capturar áreas aproximadamente quadradas e redimensionar para 640×640 pixels.

2.4. Volume e divisão do dataset

- Volume mínimo: 200 imagens com objeto-alvo por grupo (após a curadoria do mosaico).
- Diversidade geográfica: pelo menos 3 bairros diferentes no conjunto de treino.
- Cada integrante anota a sua parcela — não é permitido um único integrante anotar tudo.
- Reservar pelo menos 1 bairro inteiro fora do treino para o teste final de generalização.

3. Requisitos Técnicos

3.1. Anotação (rotulagem) das imagens

- Ferramenta obrigatória: Roboflow (recomendado) ou CVAT.ai — ambos rodam 100% no navegador.
- Desenhar bounding boxes justas em volta de cada instância do objeto.
- Definir um padrão escrito de anotação no grupo (ex.: o que fazer quando o objeto está cortado pela borda, parcialmente coberto por sombra, ou refletindo o sol).
- Todos os integrantes devem anotar — não é permitido um único integrante anotar tudo.

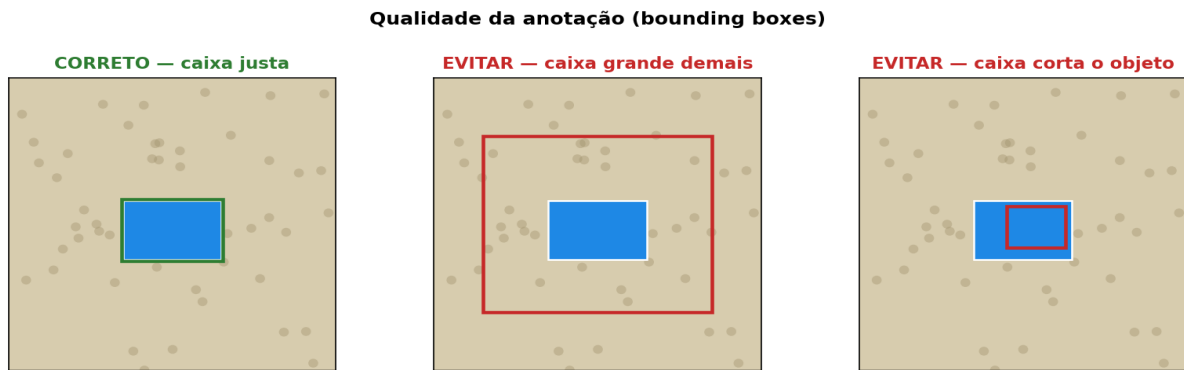


Figura 4 — Padrões de qualidade na anotação.

3.2. Pré-processamento e splits

- Resize de todas as imagens para 640×640 (configurar na própria ferramenta de anotação).
- Data augmentation no Roboflow (gerar versões adicionais sem rotular de novo): **rotação de 90°, flip horizontal e vertical**, pequenas variações de brilho/contraste.
- Divisão: 70% treino / 20% validação / 10% teste (configurável no Roboflow).
- Exportar no formato **YOLOv8 (compatível com YOLOv11)** e usar a chave de API do Roboflow no Colab para baixar o dataset.

Estrutura esperada do dataset (formato YOLOv8/11)

```
dataset/
├── data.yaml
├── train/
│   ├── images/  (.jpg / .png)
│   └── labels/   (.txt no formato YOLO)
├── valid/
│   ├── images/
│   └── labels/
└── test/
    ├── images/
    └── labels/
```

Cada .txt: uma linha por objeto → class_id x_centro y_centro largura altura
(coordenadas normalizadas entre 0 e 1)

Figura 5 — Estrutura de pastas esperada do dataset exportado.

3.3. Treinamento no Google Colab (GPU T4)

Usar a biblioteca **ultralytics** e o modelo **YOLOv8n (Nano, ~6 MB)** ou **YOLOv11n**. Esses modelos foram escolhidos por caberem confortavelmente na GPU gratuita T4 do Colab e treinarem rápido para datasets pequenos.

```
!pip install -q ultralytics roboflow

from ultralytics import YOLO

model = YOLO('yolov8n.pt')  # ou 'yolov11n.pt'

results = model.train(
    data='data.yaml',  # gerado pelo Roboflow
```

```

epochs=30,          # 30–50 épocas bastam
imgsz=640,
batch=16,           # seguro no Colab Free (reduza para 8 se faltar
memória)
device=0,           # GPU
seed=42,            # reprodutibilidade
project='runs', name='exp1'
)

```

- Fixar uma seed e registrar todos os hiperparâmetros usados.
- Salvar o melhor checkpoint (o framework já gera best.pt automaticamente em runs/detect/exp1/weights/).
- Executar pelo menos 2 rodadas variando 1 hiperparâmetro (ex.: epochs, batch ou imgsz) e comparar.

3.4. Avaliação

- Métricas obrigatórias: mAP@0.5, mAP@0.5:0.95, precisão (P) e revocação (R) no conjunto de teste.
- Matriz de confusão (gerada automaticamente pelo Ultralytics).
- Curvas de loss (box, cls, dfl) e curvas P/R por época.
- Análise qualitativa: mostrar 5 acertos claros, 3 falsos positivos e 3 falsos negativos com explicação plausível para cada erro.

Exemplo do que se espera observar ao longo do treino

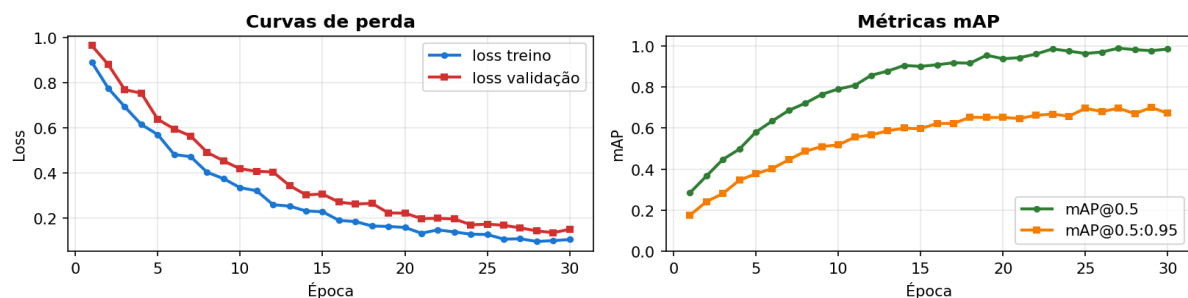


Figura 6 — Exemplo do comportamento esperado das curvas de treino e validação.

3.5. Inferência em imagens inéditas

Cada grupo deve gerar um mosaico de um bairro inteiro NÃO usado no treino e rodar o modelo sobre todos os tiles. Pelo menos 5 desses tiles devem ser discutidos no relatório (acertos, falsos positivos, falsos negativos). É nessa etapa que se verifica se o modelo realmente generaliza para uma região nova de São Paulo.

```

from ultralytics import YOLO

model = YOLO('runs/detect/exp1/weights/best.pt')
results = model.predict(
    source='imagens_ineditas/',
    save=True,
    conf=0.25
)

```

3.6. Aplicação Web (OPCIONAL — vale ponto extra)

Como diferencial, o grupo pode construir uma aplicação simples em Streamlit ou Gradio que permita ao usuário fazer upload de uma imagem de satélite e visualizar as detecções do modelo treinado. Vale até 1,0 ponto extra (a nota máxima continua sendo 10,0).

- Carregar o best.pt e expor um endpoint de upload.
- Mostrar a imagem com as caixas desenhadas e a confiança de cada detecção.

4. Entregáveis

- Link do projeto no Roboflow (ou pasta zipada do dataset anotado) com train / valid / test.
- Notebook Jupyter (.ipynb) rodando de ponta a ponta no Colab, com células de download do dataset, treino, avaliação e inferência.
- Arquivo best.pt do melhor modelo treinado.
- Pasta runs/ com logs, curvas e matriz de confusão geradas pelo Ultralytics.
- Pasta imagens_ineditas/ com as 5 imagens novas e os resultados da inferência.
- Relatório em PDF (5 a 8 páginas): introdução, definição do alvo, processo de coleta e anotação, metodologia, resultados, análise de erros, limitações e conclusão.
- Apresentação em slides (PDF ou PPTX) de no máximo 10 minutos.
- (Opcional) Código e instruções de execução do app Streamlit/Gradio.

5. Critérios de Avaliação

A nota final do trabalho (0 a 10) será composta pelos seguintes critérios:

#	Critério	Pontos
1	Coleta programática via ESRI XYZ (script, bbox e zoom documentados, curadoria do mosaico, atribuição correta)	1,5
2	Qualidade da anotação (caixas justas, consistência entre integrantes)	2,0
3	Pré-processamento e augmentation (resize, splits train/val/test)	1,0
4	Treinamento e monitoramento (curvas, hiperparâmetros, reprodutibilidade)	1,5
5	Avaliação (mAP@0.5, mAP@0.5:0.95, matriz de confusão, análise de erros)	1,5
6	Inferência em bairro inédito de SP (mosaico fora do treino)	0,5
7	Relatório técnico (clareza, justificativas, discussão de limitações)	1,0
8	Apresentação oral e domínio do conteúdo pelo grupo	1,0
—	TOTAL	10,0
+	Aplicação Web em Streamlit/Gradio (OPCIONAL — ponto extra)	+1,0

Observação: a nota máxima (incluindo o extra) é limitada a 10,0 pontos.

6. Regras e Observações

- Grupos de 3 a 5 integrantes. Trabalhos individuais só com autorização prévia do professor.
- Todos os integrantes devem saber explicar qualquer parte do trabalho na apresentação (anotação, treino, métricas, inferência).

- É permitido consultar a documentação oficial do Ultralytics, do Roboflow e tutoriais, desde que citados no relatório.
- Código copiado sem entendimento e sem citação será considerado plágio e zerará o trabalho.
- Uso de IA generativa é permitido como ferramenta de apoio, mas deve ser declarado no relatório (qual ferramenta, em que etapas, e o que foi efetivamente aproveitado).
- Imagens devem ser de áreas públicas (vista de satélite). Não anotar pessoas, placas de veículos ou qualquer dado pessoal identificável.
- Trabalhos entregues após o prazo terão desconto de 1,0 ponto por dia de atraso.

7. Dicas Finais

- Comece pela coleta e anotação imediatamente — é a parte que consome mais tempo.
- Combine no grupo um padrão escrito de anotação ANTES de começar a rotular, para evitar inconsistências entre integrantes.
- Use o batch=16 como ponto de partida; se aparecer erro de CUDA out of memory, reduza para 8.
- No Colab Free, salve o dataset e os pesos no Google Drive — a sessão expira e você perde tudo que estiver só em /content.
- Antes de entregar, rode o notebook do zero (Runtime → Restart & run all) para garantir reprodutibilidade.
- Compare pelo menos 2 experimentos no relatório (ex.: 30 vs 50 épocas, ou batch=8 vs 16).

8. Identificação do Grupo

Preencha os dados abaixo na primeira página do notebook e do relatório:

#	Nome Completo	Matrícula
1		
2		
3		
4		
5		

Bom trabalho!